AUTONOMOUS AGENT EXECUTION SPECIFICATION • PROTOCOL VER. 1.0

Project Clutch (ক্লাচ): Automated Implementation Directive

Complete engineering specification, state machines, edge-case algorithms, and sprint execution gates

Target Engine: Python (FastAPI) + Redis + PostgreSQL + Next.js	**Execution Model:** Solo Developer / "Hit-Next" Autonomous Protocol	**Security Profile:** Zero-Trust HMAC, AES-256 GCM, Strict Anti-Tamper

AUTONOMOUS "HIT-NEXT" EXECUTION DIRECTIVE: This specification is architected for step-by-step automated code generation. When the operator inputs *"Next"*, the agent must execute the subsequent phased block sequentially, complete with production-grade unit tests, hardened error handling, and zero security omissions. Every endpoint and storage layer must adhere to strict zero-trust standards to prevent token leakage, courier manipulation, and injection attacks.

1. SYSTEM ARCHITECTURE & COMPONENT HIERARCHY

Clutch enforces strict decoupling between external platform ingestion, state accumulation, parsing, and logistics dispatch. All external API calls that may suffer network timeouts are non-blocking and isolated behind distributed queues.

```
# Component Hierarchy & Event Flow [Meta Graph Webhooks] --> [FastAPI Ingestion] --> [Redis Debounce Buffer (75s TTL)] | (TTL Expire Trigger) v [Core Normalizer & Parser Engine] - Bengali Unicode Digit Converter - Phone Sanitizer (013-019 MSISDN) - 3-Tier Zone & Thana Resolver | v [PostgreSQL Database (State: PENDING)] | [Next.js Mobile Console] <-- (WebSocket / Poll) ------+ | v (Tap: "⚡ DISPATCH TO STEADFAST") [Atomic Redis Lock] --> [Celery/RQ Queue Worker] --> [Steadfast/Pathao API] | (Success: Consignment SF-XXXXXX) v [Telegram Bot Merchant Broadcast]
```

2. NORMALIZED RELATIONAL SCHEMA (POSTGRESQL)

The persistent layer must track order state transitions, unique consignments, and fraud signatures with strict idempotency keys.

TABLE	KEY FIELDS & TYPES	INDEXES, CONSTRAINTS & BUSINESS RULES
`merchants`	`id` (UUID), `page_id` (VARCHAR), `access_token` (TEXT, ENC), `sf_api_key` (TEXT)	Unique index on `page_id`. Token health monitor validates token validity hourly.
`orders`	`id` (UUID), `merchant_id`, `sender_id`, `raw_text`, `clean_phone` (VARCHAR 11), `address`, `zone`, `cod_amount` (DECIMAL), `status`	Unique composite index: `(merchant_id, sender_id, created_at::date)`. Status: `pending`, `needs_review`, `queued`, `dispatched`, `cancelled`.
`consignments`	`id`, `order_id`, `courier` (ENUM), `tracking_code`, `consignment_id`, `pickup_status`	Unique index on `consignment_id`. Foreign key cascade to `orders.id`. Prevents duplicate booking.
`trx_audit_log`	`id`, `trx_id` (VARCHAR 16), `provider` (bKash/Nagad), `order_id`, `verified_at`	Unique constraint on `trx_id`. Blocks duplicate TrxID reuse fraud across all merchant accounts.

3. MANDATORY ALGORITHMIC GUARDRAILS (AGENT CODE TASKS)

The autonomous agent must implement these exact algorithmic functions within the core ingestion worker:

A. Multi-Message Debounce Buffer (Redis)

When a customer sends short message bursts, append incoming payloads to a list: `session:{page_id}:{sender_id}`. Reset TTL to 75 seconds. Once the TTL expires with no new incoming messages, pop all items, concatenate into a single text block, and pass to the parser.

B. Bengali Unicode Digit Normalizer & Phone Sanitizer

```
def normalize_bd_phone(raw_text: str) -> list[str]: # 1. Map Bengali digits (০-৯) to ASCII (0-9) bn_map = str.maketrans("০১২৩৪৫৬৭৮৯", "0123456789") ascii_text = raw_text.translate(bn_map) # 2. Extract 11-digit numbers with BD prefixes (013-019) sanitized = re.sub(r"[\s\-\+\(\)]", "", ascii_text) matches = re.findall(r"(?:8801|01)[3-9]\d{8}", sanitized) return [m[-11:] for m in matches] # Normalize to standard 11 digits
```

C. 3-Tier Delivery Zone & Suburb Classifier

Evaluate raw text against geographic dictionaries before assigning courier charges:

- **Inside Dhaka (৳70):** Match core zones (Uttara, Mirpur, Dhanmondi, Gulshan, Mohammadpur, Badda, Banani, Motijheel, etc.).
- **Dhaka Suburbs (৳100):** Match boundary towns: *Savar, Gazipur, Tongi, Keraniganj, Narayanganj, Ashulia*.
- **Outside Dhaka (৳130):** Match any other of the 64 districts or default fallback.

D. Human Takeover Suppression Sensor

Intercept Meta webhook payloads where `entry.messaging.message.is_echo == True` or sender is Page Admin. Immediately execute: `redis.set(f"takeover:{page_id}:{sender_id}", 1, ex=2700)` (45-min TTL). The bot must not emit any automated response while this key exists.

E. Double-Click Dispatch Lock

Wrap courier booking in an atomic distributed lock: `lock = redis.set(f"lock:dispatch:{order_id}", "locked", nx=True, ex=30)`. If acquisition fails, abort and return `{"status": "already_processing"}` to prevent duplicate courier consignments.

4. ENTERPRISE SECURITY & ANTI-EXPLOIT ARCHITECTURE

Because Clutch handles sensitive customer PII (phone numbers, home addresses), merchant financial balances, and Meta page management tokens, the system implements a strict multi-layer security posture:

VULNERABILITY VECTOR	ATTACK MECHANISM / THREAT	MANDATORY HARDENING & MITIGATION STANDARD
Webhook Spoofing	Malicious actor injects forged Meta comment/DM payloads to drain API quotas or push fake orders.	**HMAC-SHA256 Verification:** Validate raw payload against `X-Hub-Signature-256` header using merchant's App Secret before JSON parsing. Reject on mismatch.
Credential & Token Theft	Database dump exposes long-lived Meta Page Access Tokens and Steadfast API keys.	**AES-256-GCM Envelope Encryption:** All 3rd-party tokens stored at rest encrypted via AES-256-GCM with keys isolated in environment variables.
API Abuse & DoS	Bot floods the public webhook endpoint to exhaust server memory and trigger rate limits.	**Redis Token Bucket Rate Limiting:** Enforce strict 60 req/min limit per IP; edge-proxy Cloudflare rules blocking non-Meta IP ranges on `/webhooks`.
Injection & Script Poisoning	Attacker injects SQL payloads or XSS vectors through conversational Facebook DM fields.	**Parameterized Queries & Strict Typings:** Enforce Pydantic v2 input sanitization and SQLAlchemy prepared statements. Reject raw SQL strings entirely.
IDOR & Consignment Hijacking	Merchant A attempts to inspect or cancel orders belonging to Merchant B by guessing order UUIDs.	**Tenant Isolation:** Every query enforces a hard `WHERE merchant_id = current_user.tenant_id` clause validated via signed JWT session cookies.

5. MERCHANT CONSOLE UI TOKEN SPECIFICATIONS

The Next.js/Tailwind frontend must strictly implement the Cyber-Magenta & Deep Obsidian design system with high-contrast text and uncrowded 24px spatial margins:

- **Canvas Void:** `#0E0A12` • **Surface Slate:** `#16101D` • **Border Wire:** `#341B3B`
- **Primary Kinetic Accent:** `#FF1493` (Hot Cyber Magenta) — Reserved strictly for the single dispatch trigger.
- **Active Automation State:** `#00F5A0` (Acid Mint) — Webhook listening, API online, consignment confirmed.
- **Warning / Fraud State:** `#F59E0B` (Signal Amber) — Return risk >40%, advance payment unverified.

6. PHASED "HIT-NEXT" SEQUENTIAL BUILD PROTOCOL

The autonomous agent progresses through the following sequential blocks. Each step must compile, pass tests, and satisfy all security constraints before the operator enters *"Next"*:

STEP	COMPONENT NAME	KEY IMPLEMENTATION DELIVERABLES	AUTOMATED VALIDATION GATE
Step 1	Sanitizer & HMAC Gate	Bengali digit converter, phone regex, zone classifier, and Meta HMAC-SHA256 signature verifier.	100% pass on 20 dirty Banglish strings; unauthenticated payloads return 401.
Step 2	Buffer & Takeover Engine	75s Redis debounce accumulator, Admin Takeover sensor (45-min bot suppression), and 24h window monitor.	Multi-message bursts group cleanly; bot instantly silences upon admin reply.
Step 3	Encrypted Schema & Queue	PostgreSQL schema with AES-256 token fields, Celery retry worker, and atomic Redis dispatch lock.	Simulated 504 outage triggers clean retry; duplicate clicks yield single booking.
Step 4	Courier API Adapters	Steadfast & Pathao API clients, Thana/District offline mapper, and bKash TrxID deduplication filter.	Valid consignment ID generated; duplicate TrxIDs rejected automatically.
Step 5	Cyber-Magenta UI Console	Next.js 14 console (#0E0A12 / #FF1493), side-by-side verification cards, single-tap dispatch button.	Actionable UI loads <1.5s on mobile; single-tap booking updates state optimistically.